Day 1

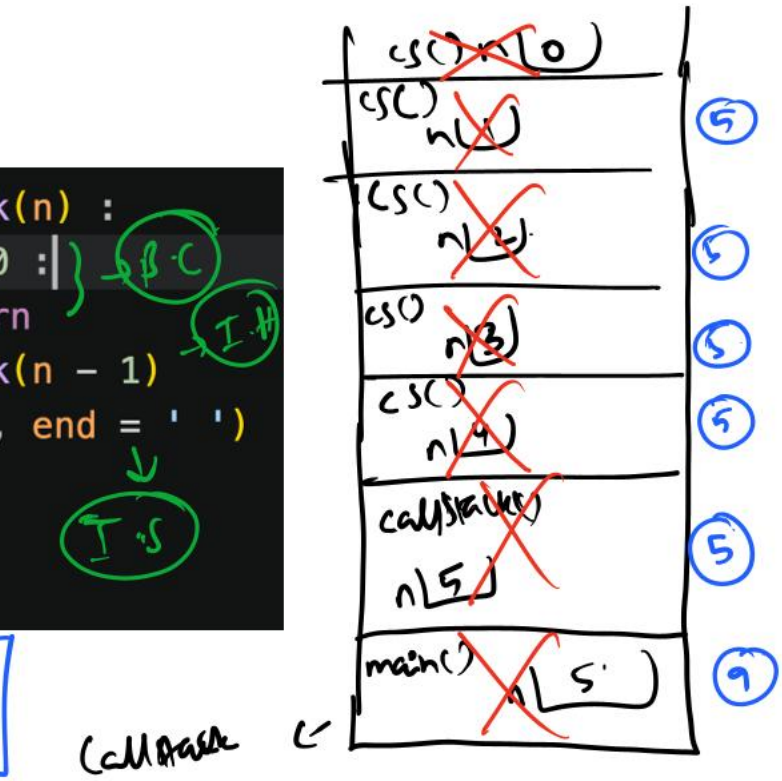
Monday, 2 November 2026 9:52 AM

```
2  def callStack(n) :
3      if n == 0 :
4          return
5      callStack(n - 1)
6      print(n , end = ' ')
7
8  n = 5
9  callStack(n)
```

B.C

I.H

I.S



1 2 3 4 5

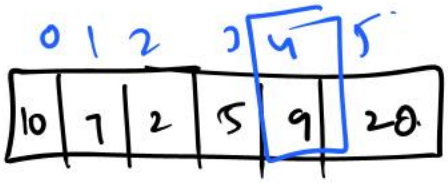
Console.

CallStack

EMPTY.

10^4

Functions



1sec → 10^4 ops

2sec → 10^8 ops

$1 \le n \le 10^5$

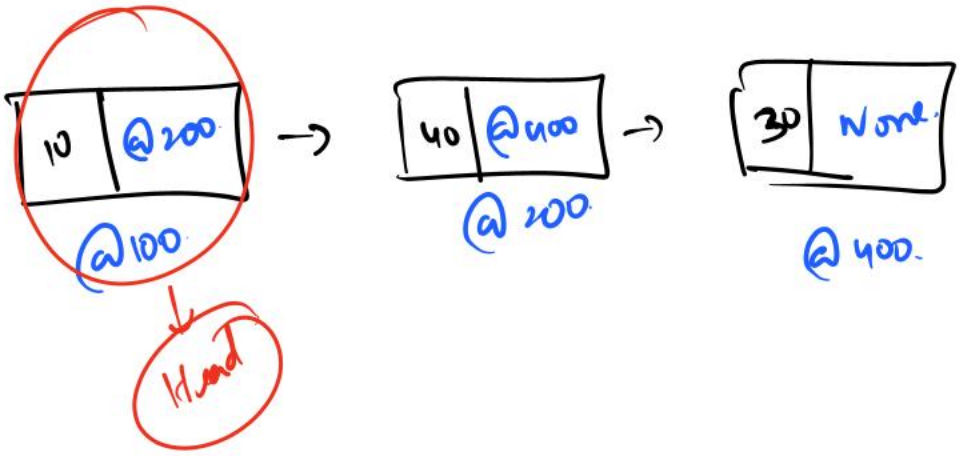
T.C → $O(n^2)$

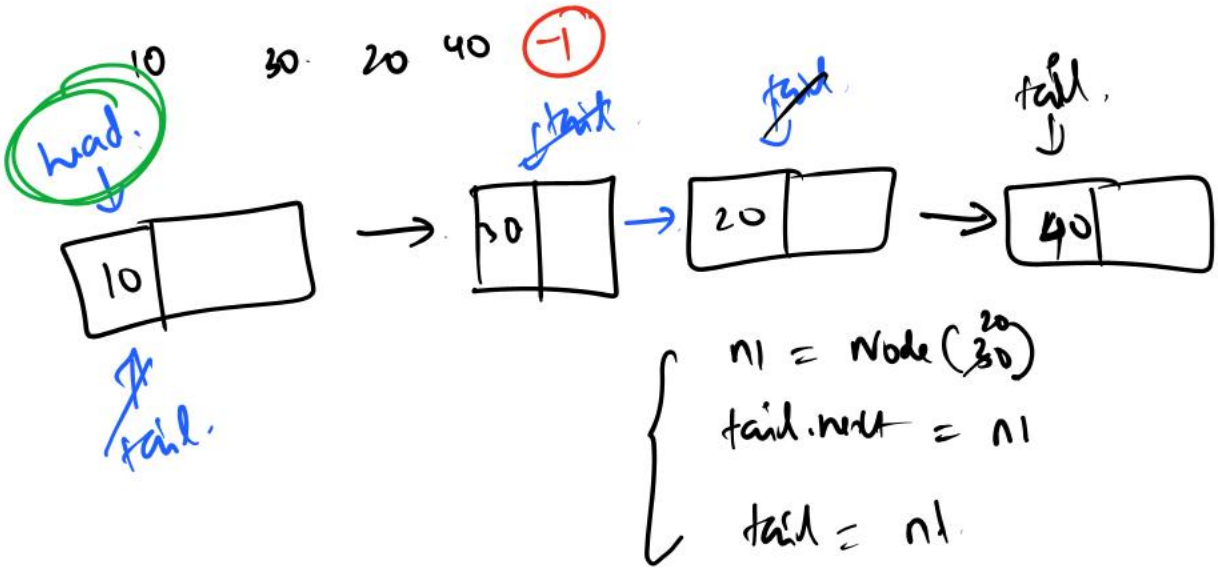
$(10^5)^2 \rightarrow 10^{10} > 10^8$

TLE

10^8 ops

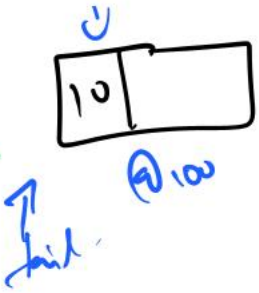
	n
$O(n)$	10^8
$O(n^2)$	10^4
$O(n \log n)$	?
$O(\log n)$	?
$O(2^n)$	?





head

@100

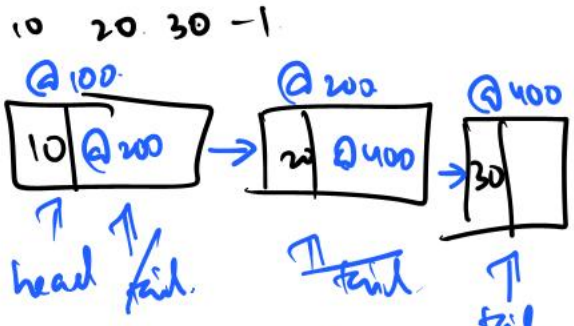








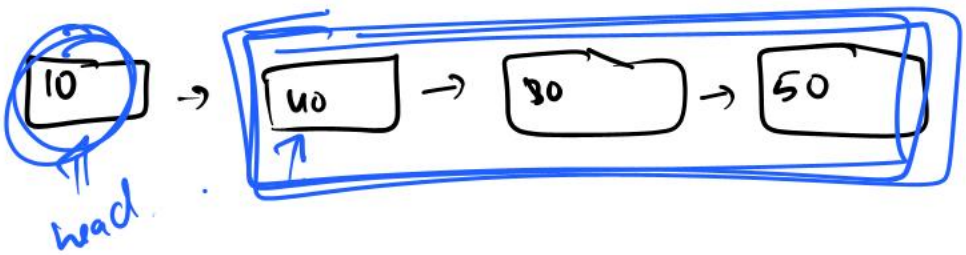

```
13  def buildList() :
14      head, tail = None, None
15      while True :
16          data = int(input('Enter data : '))
17          if data == -1 :
18              return head
19          node = Node(data)
20          if head is None :
21              head = tail = node
22          else :
23              tail.next = node
24              tail = node
25      return head
```















Problem → Depending on similar sub problems → RECURSION

Recursion → PMI → Principle of Mathematical Induction

$\frac{n(n+1)}{2}$ ① → $\Sigma 1 = 1$

→ Assume $\Sigma k = \frac{k(k+1)}{2}$

↓

$$\Sigma k+1 = \frac{(k+1)(k+2)}{2}$$

$= \Sigma k+1$

$= \Sigma k + k+1$

$= \frac{k(k+1)}{2} + k+1$

$= \frac{(k+1)}{2}(k+2)$

$= \frac{k(k+1) + 2(k+1)}{2}$

PMI

① Smallest solution.

```
27  def printList(head) :
28      if head is None :
29          return
30      print(head.data , end = ' -> ')
31      printList(head.next)
```

→ Base case

(2) Assumption

(1) Step

Induction Step

Induction Hypothesis

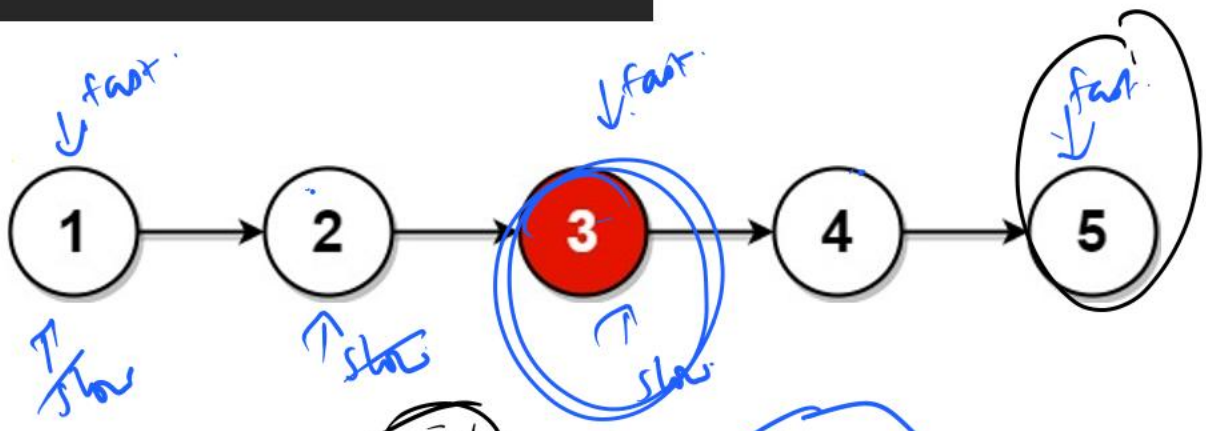
876. Middle of the Linked List

Solved

Easy | Topics | Companies

Given the head of a singly linked list, return *the middle node of the linked list*.

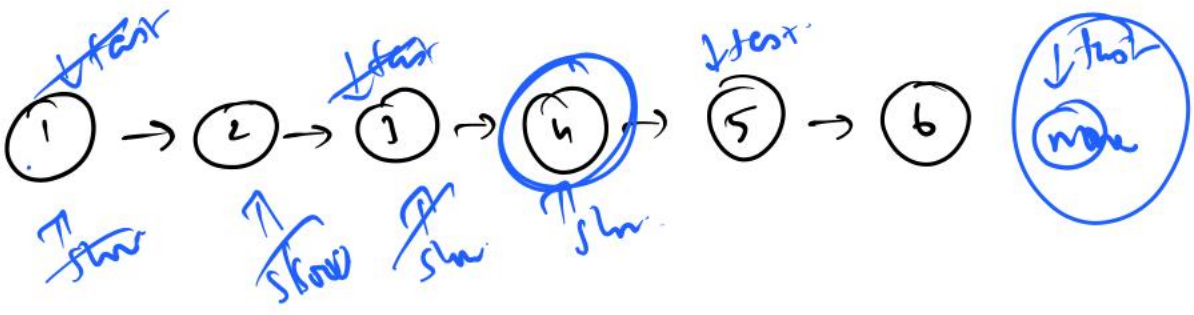
If there are two middle nodes, return **the second middle** node.



Arithmetic Flow

fast = fast.next.next

slow = slow.next



.. next

fast = fast.next.next

slow = slow.next

```python
class Solution:
    def middleNode(self, head: Optional[ListNode])
    -> Optional[ListNode]:
        fast , slow = head, head
        while fast and fast.next :
            fast = fast.next.next
            slow = slow.next
        return slow
```

T.C -> O(n)

S.C -> O(1)

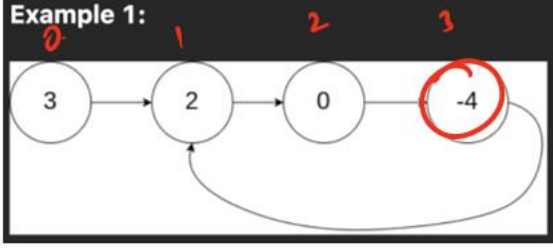
P.NO: 876

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

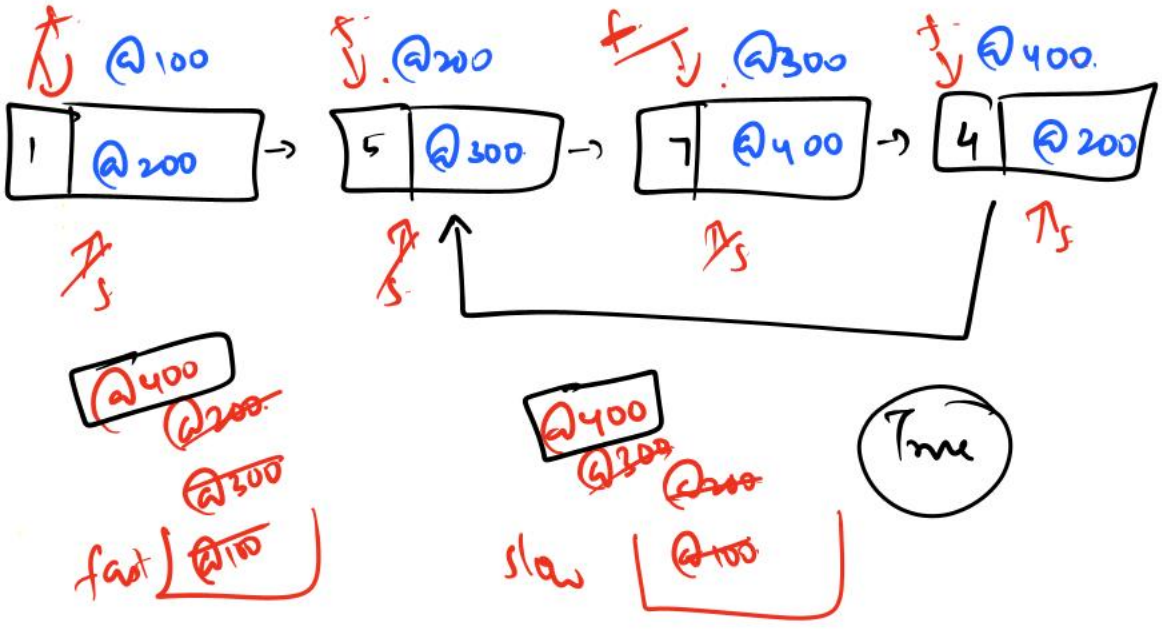
Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

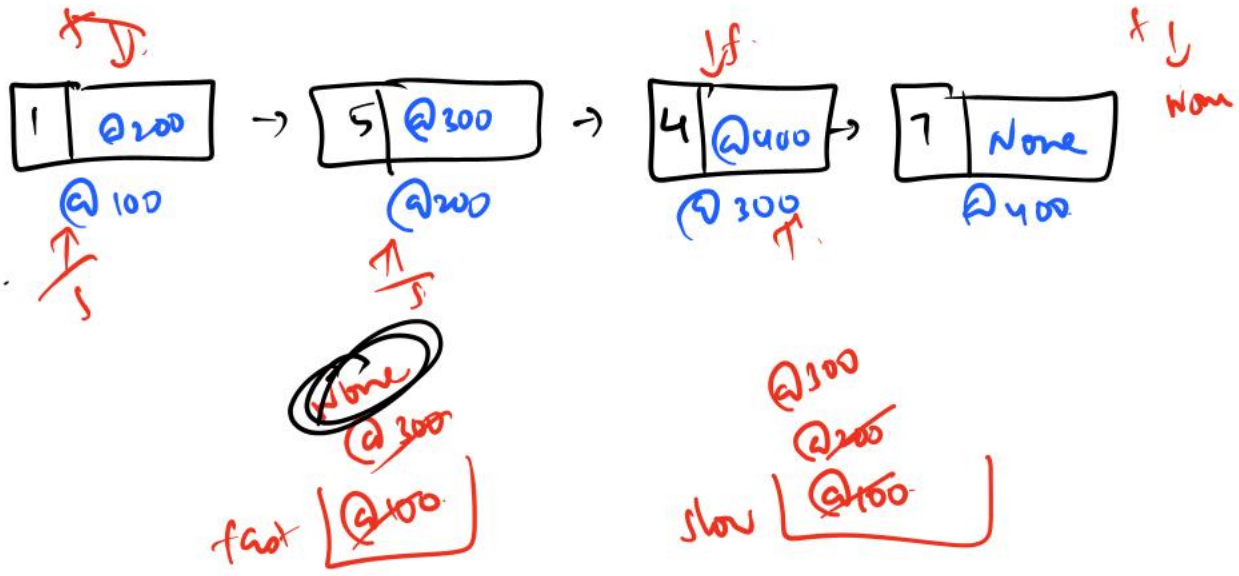
Example 1:

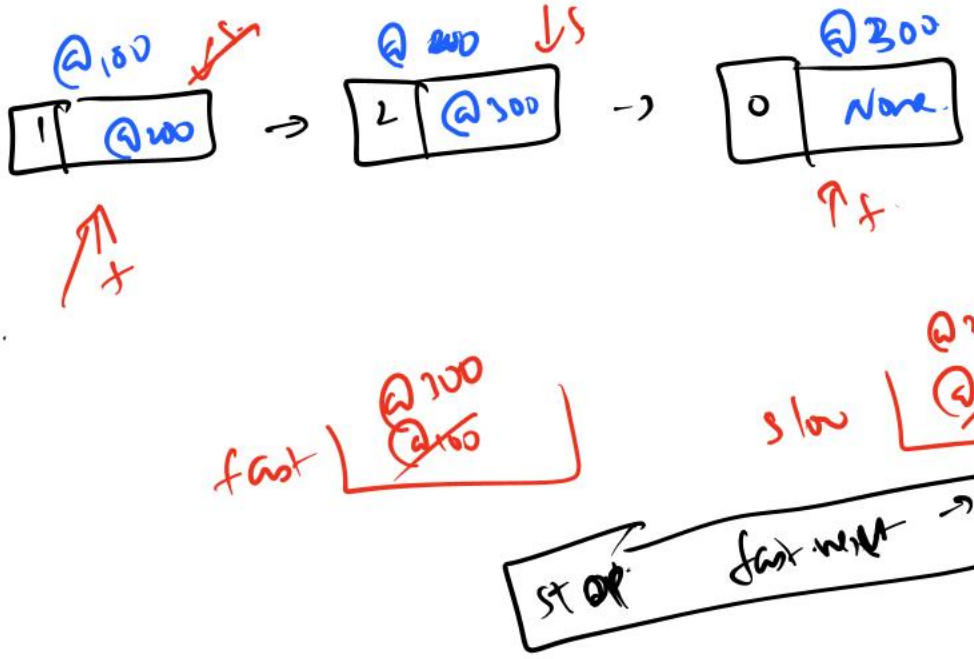


P.NO:

141


@100
@200
@300
@400
1 @200
5 @300
7 @400
4 @200
fast @400 @300 @100
slow @400 @200 @100
True

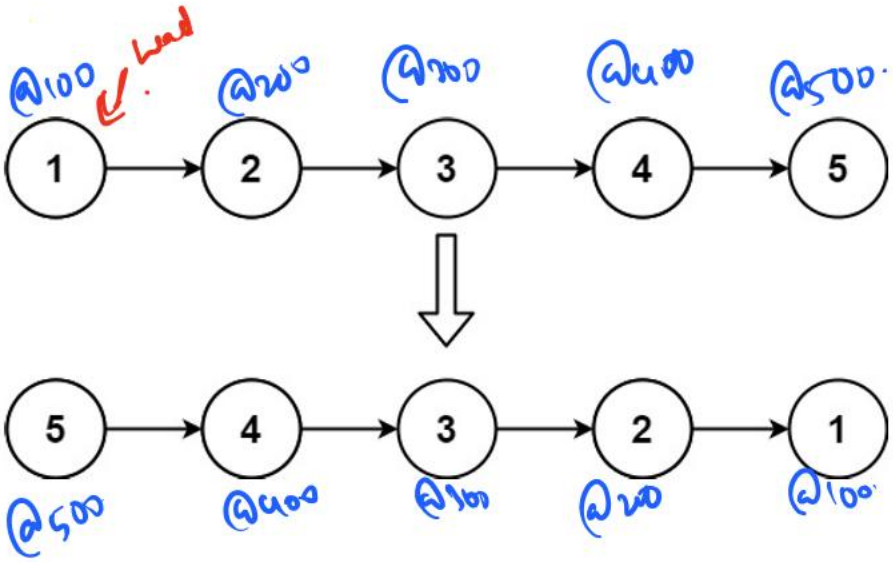

1 @200
5 @300
4 @400
7 None
@100
@200
@300
@400
None
fast @400
slow @100 @200 @400

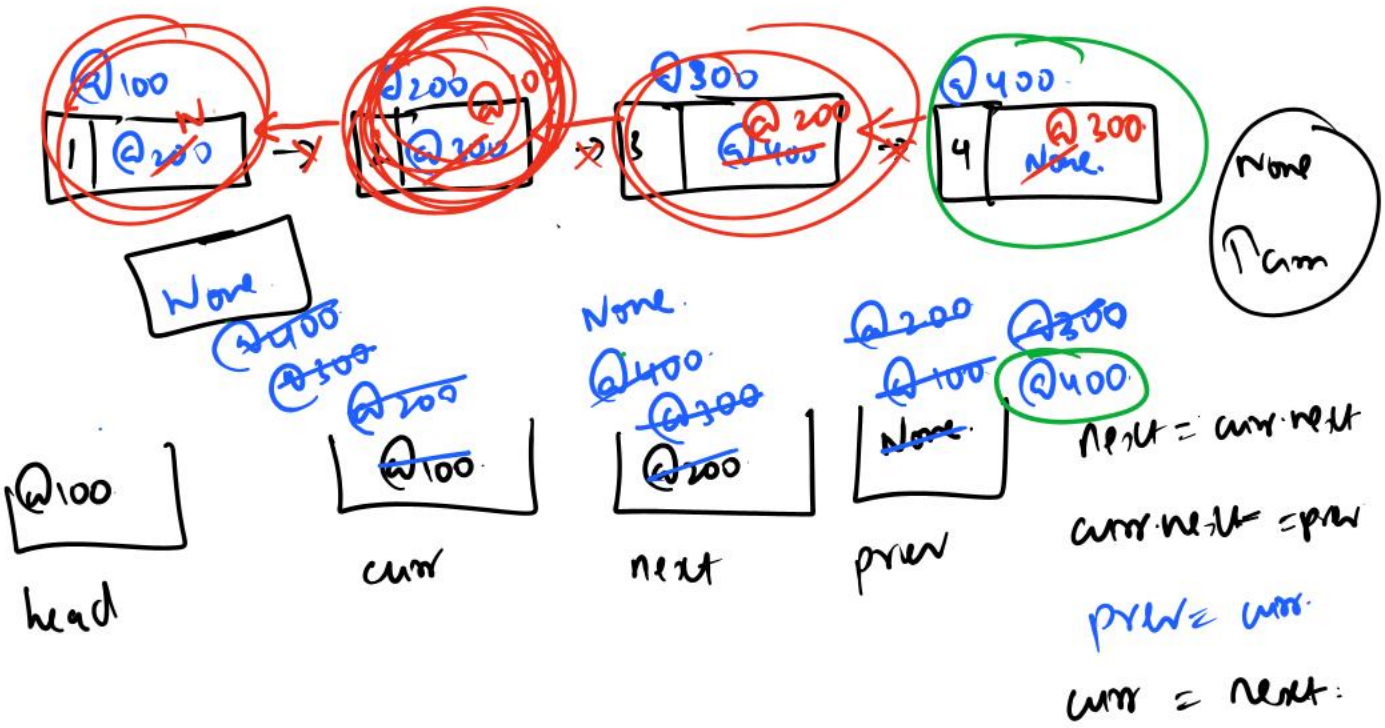

@100
@200
@300
1 @200
2 @300
0 None
fast @300 @100
slow @200 @400
stop fast will → now

```python
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        fast = slow = head
        while fast and fast.next :
            fast = fast.next.next
            slow = slow.next
            if fast is slow :
                return True
        return False
```

T.C -> O(n)

S.C -> O(1)

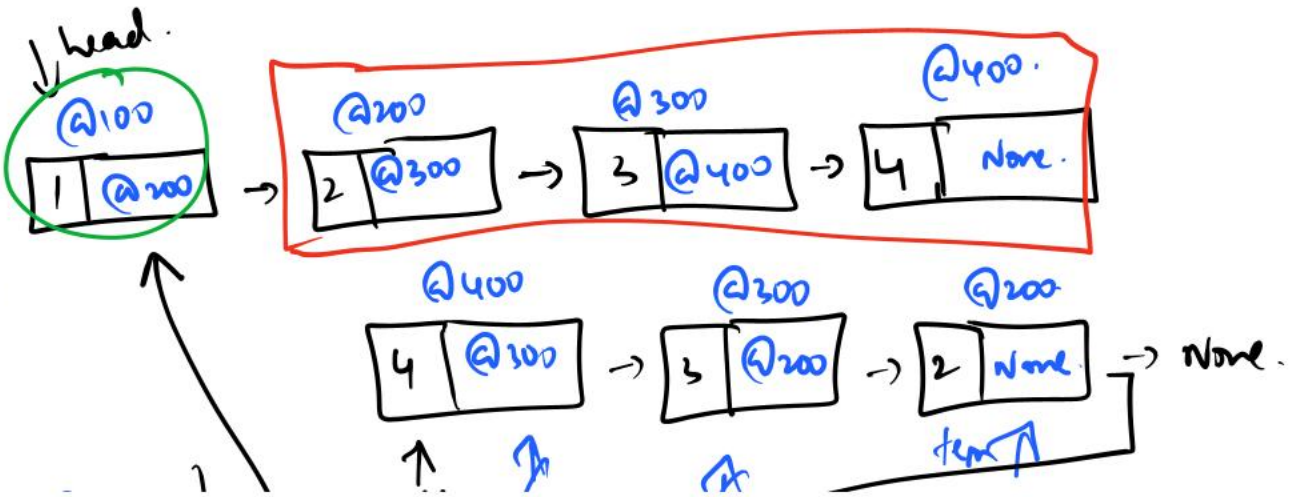


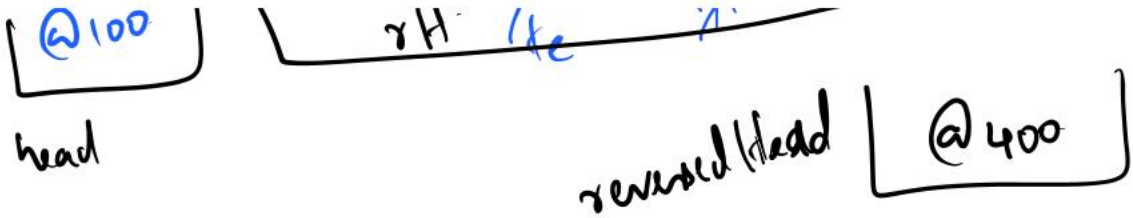



```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        curr , prev = head, None
        while curr :
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next
        return prev
```

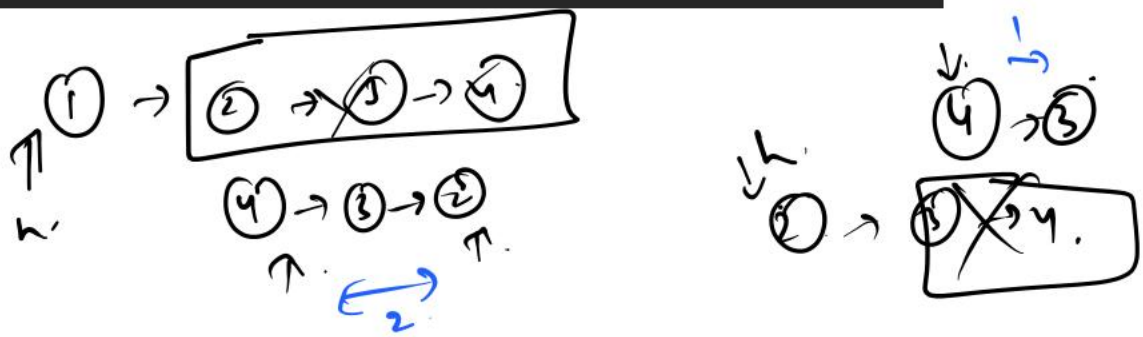
T.C -> O(n).

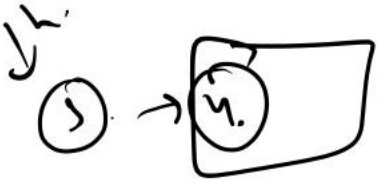
S.C -> O(1).





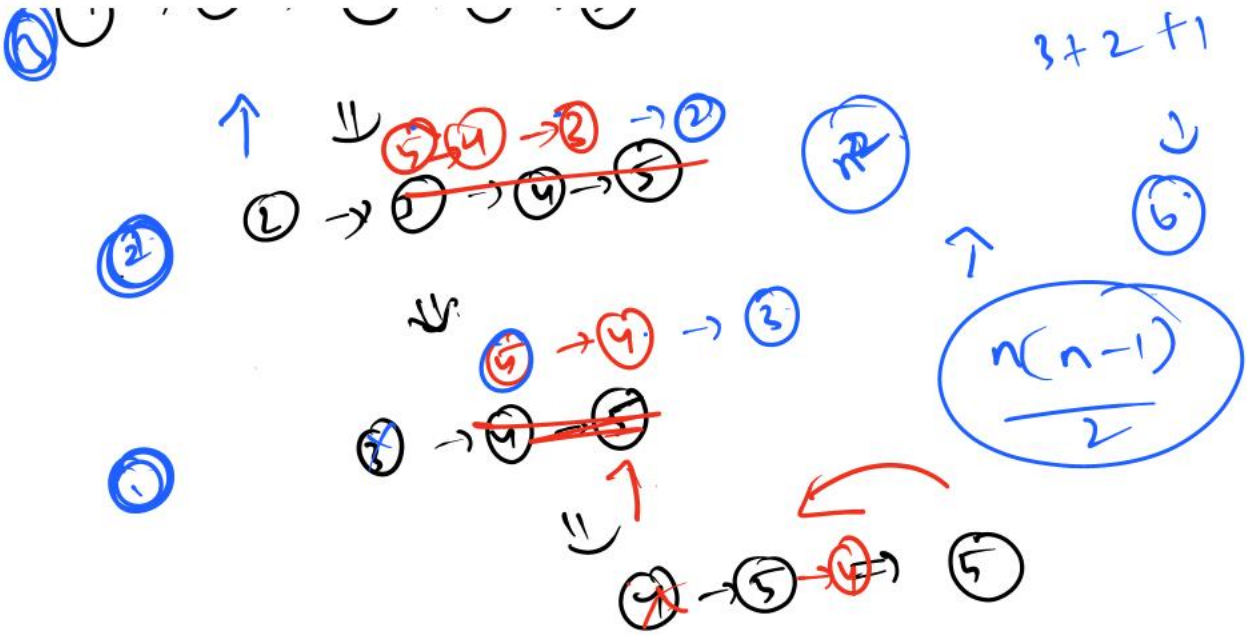

```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional
    [ListNode]:
        if head is None or head.next is None :
            return head
        reversedHead = self.reverseList(head.next)
        temp = reversedHead
        while temp.next :
            temp = temp.next
        temp.next = head
        head.next = None
        return reversedHead
```

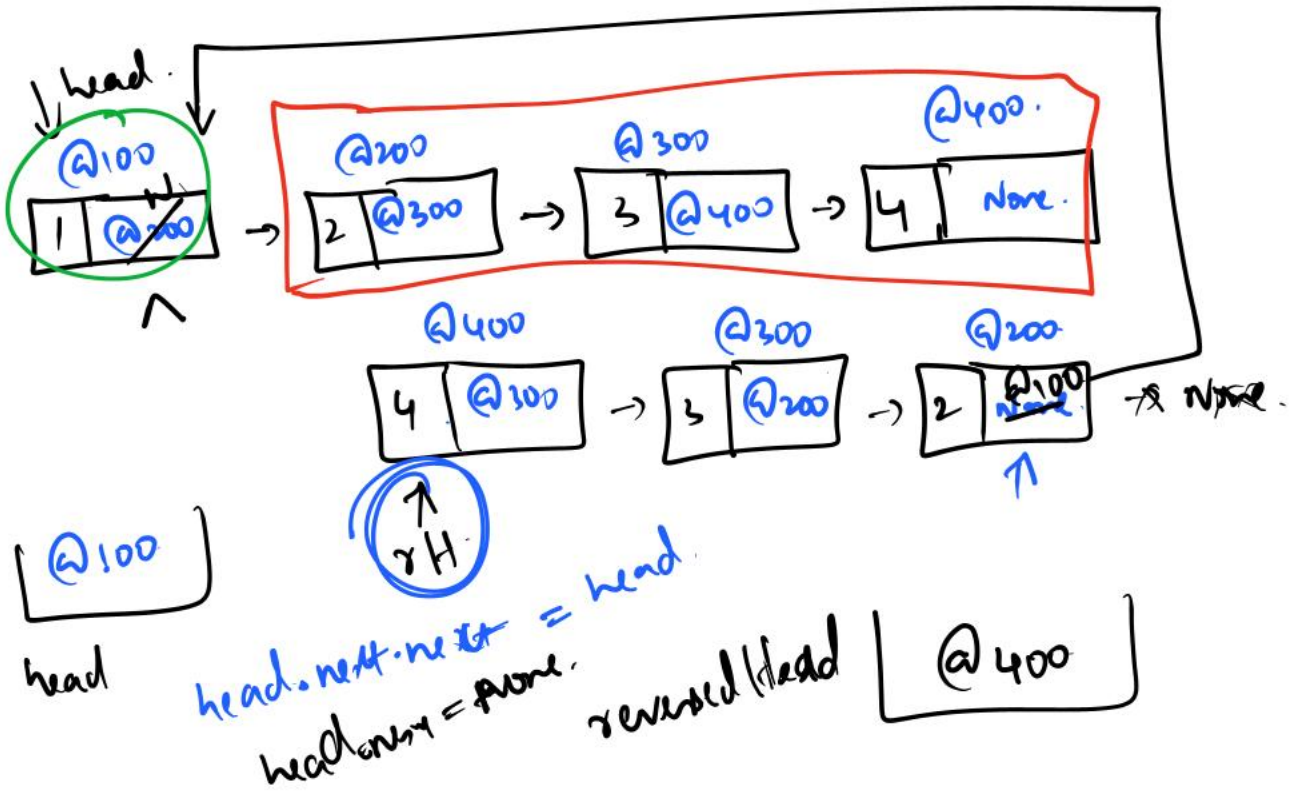









3+2+1
n(n-1)/2


head
@100
1
@200
@200
2
@300
@300
3
@400
@400
4
None
@400
4
@300
@300
3
@200
@200
2
@100
None
rH
@100
head
head.next.next = head
head.next = None
reversed Head
@400

```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None :
            return head
        reversedHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None # head is last Node, last node's next -> None
        return reversedHead
```

T.C → O(n)

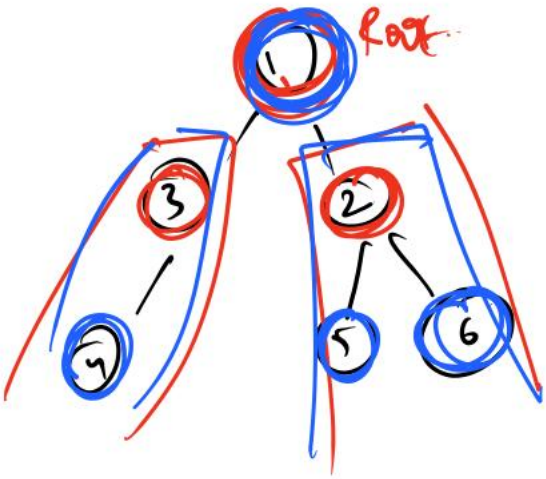
S.C → O(n)

```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional
    [ListNode]:
        if head is None or head.next is None :
            return head
        reversedHead = self.reverseList(head.next)
        temp = reversedHead
        while temp.next :
            temp = temp.next
        temp.next = head
        head.next = None
        return reversedHead
```

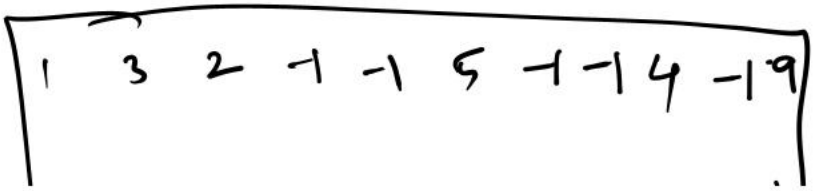
T.C → O(n)

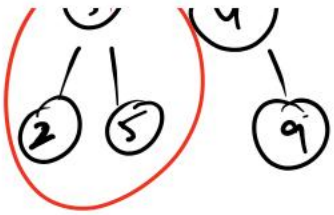
S.C → O(1)

Binary Trees



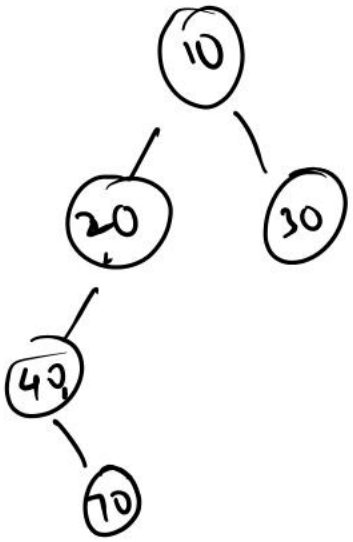






-1 -1

1 3 2 5 4 9.



10 20 40 70 30

10 : L 20 R : 30

20 : L 40

40 : R 70

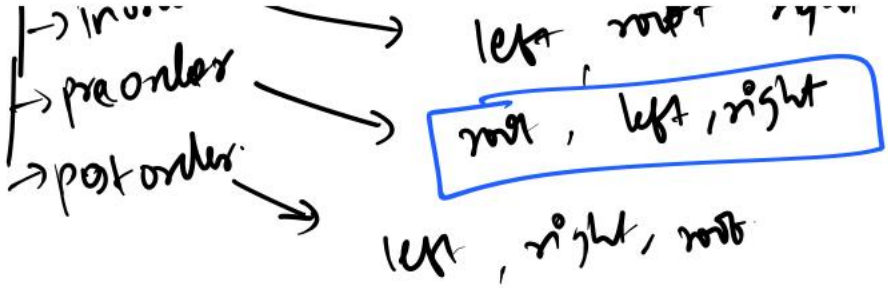
70 :

30 :

Traversal

DFS

BFS




```python
class Solution:
    def inorderTraversal(self, root: TreeNode | None) -> list[int]:
        def helper(root) :
            if root is None :
                return
            helper(root.left)
            ans.append(root.val)
            helper(root.right)
        ans = []
        helper(root)
        return ans
```

T.C → O(n)

S.C → O(n)

call stack space.